

User Manual: Personal Information Management System

Course: COMP3211 Software Engineering, Fall 2026

Group: 89

Member names and student IDs: Pending group-provided data

Product baseline: 4d3508db0447a3ac6346a2a2789da89208f88fd5

Document version: WP05 final document, 2026-09-30

1. What the program does

The Personal Information Management (PIM) program keeps four kinds of personal information record (PIR): notes, tasks, events, and contacts. You can create, list, inspect, change, search, and delete them at a command prompt. You can save the whole collection to a `.pim` file and load it in a later run. The program is local and uses text commands; an event's alarm time is stored information, not an active notification.

Each new PIR receives a positive integer ID. IDs start at 1 in a new empty process, increase after successful creation, and are not reused after deletion. A record's ID and type cannot be changed. Duplicate field values are allowed. A failed creation does not consume an ID.

2. Start, prompt, and exit

Open PowerShell in the repository root and run:

```
python src/main.py
```

The program writes `pim>` and waits for one command per line. Type `help` to display the built-in command summary. Type `exit` to finish. End-of-file also exits normally. Exiting does **not** save automatically: run `save` first if you want to keep the current collection for a later process. Every new process starts empty until you load a saved file.

The WP04 freeze was run on Windows 11 with Python 3.12.3 and 3.13.5. Python 3.11 was not available for verification and is not claimed as tested.

3. Input rules and record fields

Commands, type names, and field names ignore letter case. Separate command arguments with whitespace. Enclose a value or path containing spaces in double quotes; a one-token non-search value can be unquoted. Inside a quoted argument, `\` represents a literal double quote and `\\` represents a literal backslash. Other backslash escapes are invalid. A quoted argument must be followed by whitespace or the end of the command. Search expressions have their own quoting rules in §5.

All record fields are required, must be nonempty after leading and trailing whitespace is trimmed, and must contain a single line of valid UTF-8 text. Stored text is the trimmed value. A mobile number is text, so `+` and leading zeroes are retained; the program does not apply a regional phone-number format. Time fields use local 24-hour `YYYY-MM-DD HH:mm` at minute precision, for example `2026-10-01 09:00`. Impossible calendar dates and other time formats are rejected. Past dates are allowed, and an event's `alarm_time` need not precede its `start_time`.

Type	add field order	Fields shown by show
note	text	text
task	description, deadline	description, deadline
event	description, start_time, alarm_time	description, start_time, alarm_time
contact	name, address, mobile_number	name, address, mobile_number

4. Command reference

TYPE is one of the four types above. ID is a positive decimal integer. FIELD is a data field belonging to that record's type. Quoted forms below also work for one-token values.

Command	Format and example	Result
Help	help	Prints all command forms, field order, quoting, time, and search rules.
Create note	add note "memo"	Creates a note; prints Created note ID.
Create task	add task "draft" "2026-10-01 09:00"	Creates a task; prints Created task ID.
Create event	add event "meeting" "2026-10-02 10:00" "2026-10-01 08:00"	Creates an event; prints Created event ID.
Create contact	add contact "Li" "Room 1" "+00123"	Creates a contact; prints Created contact ID.
List all or one type	list or list task	Prints one summary per PIR in ascending ID order. A type argument filters the list.
Show one or all	show 2 or show all	Prints the ID, type, and every data field. show all is in ascending ID order.
Change one field	update 2 description "revised"	Replaces one field on an existing PIR; prints Updated 2. Use the exact field name for that PIR type.
Delete one	delete 2	Removes the specified PIR; prints Deleted 2. Its ID is not reused.
Search	search type = "task"	Prints matching summaries in ascending ID order. Full grammar is in §5.
Save	save "snapshot.pim"	Writes the full collection and next-ID counter; prints Saved N PIR(s) to PATH.
Load	load "snapshot.pim"	Replaces the current collection with the file's records; prints Loaded N PIR(s) from PATH.
Exit	exit	Ends the process without an implicit save.

help and exit take no arguments. add needs exactly the fields listed in the table and in that order. update changes exactly one field per command. list accepts at most one type. show, delete, save, and load each require exactly one argument. search requires an expression after the command word. Blank input simply returns to the next prompt.

Reading output

list and search display ID | type | first text field on each line. The first text field is text for a note, description for a task or event, and name for a contact; the entire stored value is shown. show prints id: and type: followed by all data-field labels and values. show all separates records with a blank line. An empty list or show all prints No PIRs.; an empty search prints No matching PIRs.. These messages are different so you can distinguish an empty collection/list filter from a search with no result.

5. Search expressions

A search command defines and applies its criterion at once. Every search value, **including a one-word type or text value**, must be in double quotes. Quoted search literals support only \" and \\ escapes. Search field names, contains, and type names ignore letter case. Text containment also ignores letter case, matches a substring, and requires a nonempty quoted string. Time comparisons use the exact local time format from §3 and compare chronological values.

Criterion	Valid form	Example
PIR type	type = "TYPE"	type = "contact"
Text containment	TEXT_FIELD contains "STRING"	address contains "Room"
Time comparison	TIME_FIELD < "TIME", >, or =	deadline < "2026-10-02 00:00"

TEXT_FIELD is one of text, description, name, address, or mobile_number. TIME_FIELD is one of deadline, start_time, or alarm_time. Every time field supports each of <, >, and =. A valid field absent from a particular PIR evaluates false for that PIR; negating that condition evaluates true. An unknown field is an error for the whole search.

Combine complete criteria with && (AND), || (OR), ! (NOT), and parentheses. Parentheses bind first, then !, then &&, then ||; operators at the same precedence associate left to right. Spaces around symbolic operators and parentheses are optional when the token boundary is clear. For example:

```
search (type = "task" && deadline < "2026-10-02 00:00") || text contains "memo"
search !name contains "Li"
search start_time = "2026-10-02 10:00"
```

The first line is the verified WP04 search in §8. The other forms follow the frozen parser and search tests. A malformed expression, wrong operator, invalid type or time value, empty contains string, or unmatched parenthesis produces Error: SEARCH: ... and no partial search result.

6. Saving, loading, and paths

Use a filename ending in the exact lowercase suffix .pim; .PIM and other suffixes are rejected. Quote a path containing spaces. A relative path is resolved against the working directory from which this program was started. The success message shows the resolved path. An absolute path is used as entered.

save writes all current PIRs, including an empty collection, and the next-ID counter as a UTF-8 version-one .pim snapshot. Saving to an existing destination replaces that file after a complete temporary write. A failed save does not change the in-memory records; if the destination already exists, the failed write leaves its prior bytes unchanged under ordinary handled I/O failure. Save does not happen automatically when exiting.

load reads and validates the **whole** .pim file before replacing the current collection. It replaces rather than merges records. If it fails, the active collection and next-ID counter remain intact. After a successful load, a new PIR uses the higher of the session's former next ID and the file's saved next ID. This prevents reuse of an ID already issued in the current session, even when loading an older snapshot. A fresh process only knows the counter present in the loaded file.

The .pim file is a program snapshot, not a free-form text import. The loader rejects unreadable files, invalid UTF-8 or JSON, unsupported format/version, duplicate or unexpected keys, invalid fields or values, duplicate or unordered IDs, and an invalid next-ID counter. Editing the file manually can make it unloadable.

7. Invalid input and recovery

Handled errors use Error: CATEGORY: DETAIL and then show another pim> prompt. The detail identifies the rejected input or failure and may include an operating-system file error. Correct the command and retry; a handled failure does not partially change the in-memory PIR collection.

Category	Typical cause	What to check
COMMAND	Unknown command, wrong argument count, malformed quote/escape, or a nonpositive/nondigit/overlong ID token	Command form, spaces, quotes, and positive decimal ID.
VALIDATION	Unknown PIR type, empty or multiline field, invalid field for update, or impossible/wrongly formatted time	Field order and spelling, required text, and YYYY-MM-DD HH:mm.
NOT_FOUND	A valid positive ID is absent	Use list to see current IDs.
SEARCH	Missing or malformed criterion, invalid search field/type/operator/literal	Quote each literal and check the grammar in §5.
PATH	Path lacks exact lowercase .pim suffix	Use a .pim filename.
FORMAT	Existing .pim content is invalid or unsupported	Load a snapshot written by this program.
IO	File is missing, directory is unavailable, or the operating system refuses a read/write	Check the resolved path and filesystem access.

For example, WP04's CLI tests verify that an invalid update leaves the old value visible, a failed load leaves the active collection visible, and the next command still runs. The frozen executable also produced Error: VALIDATION: text must not be empty for add note "", Error: COMMAND: invalid positive decimal ID: '0' for show 0, and Error: NOT_FOUND: record ID 99 was not found for show 99. Exact operating-system wording after an IO error depends on the path and machine.

8. Complete two-process example

This is the WP04 manual CLI acceptance transcript from docs/reports/integration-test-log.md, recorded against the freeze commit. Both Python processes exited with code 0; the snapshot existed. The evidence file replaced only the absolute temporary directory with <TEMP_DIR> below. That token is a redaction of the test location, not text to type as a path. Run both sessions from the **same** working directory so the relative snapshot.pim resolves to the same file.

First start python src/main.py, then enter:

```
add note "memo"
add task "draft" "2026-10-01 09:00"
add event "meeting" "2026-10-02 10:00" "2026-10-01 08:00"
add contact "Li" "Room 1" "+00123"
list
search (type = "task" && deadline < "2026-10-02 00:00") || text contains "memo"
update 2 description "revised"
delete 1
save "snapshot.pim"
exit
```

Recorded output:

```
pim> Created note 1
pim> Created task 2
pim> Created event 3
pim> Created contact 4
pim> 1 | note | memo
2 | task | draft
3 | event | meeting
4 | contact | Li
pim> 1 | note | memo
2 | task | draft
pim> Updated 2
pim> Deleted 1
pim> Saved 3 PIR(s) to <TEMP_DIR>\snapshot.pim
pim>
```

Start python src/main.py again from the same directory and enter:

```
load "snapshot.pim"
show all
add note "after load"
show 5
exit
```

Recorded output:

```
pim> Loaded 3 PIR(s) from <TEMP_DIR>\snapshot.pim
pim> id: 2
type: task
description: revised
deadline: 2026-10-01 09:00

id: 3
type: event
description: meeting
start_time: 2026-10-02 10:00
alarm_time: 2026-10-01 08:00

id: 4
type: contact
name: Li
address: Room 1
mobile_number: +00123
pim> Created note 5
pim> id: 5
type: note
text: after load
pim>
```

The loaded collection contains IDs 2–4 because ID 1 was deleted before saving. The next creation gets ID 5, confirming that deletion and reload did not reuse ID 1.